

Ejercicio entregable – "Organizando Información con Estructuras de Datos"

Actividad 1 - práctica

Redictado Taller de Lenguajes - Segundo Semestre 2026

Contexto

En futuros trabajos, vas a trabajar con datasets reales que tienen muchas columnas. Para poder entender y manipular esos datos de manera eficiente, es fundamental aprender a organizar la información de las columnas usando estructuras de datos adecuadas. También será importante poder configurar qué columnas mostrar según el rol de quien consulta (por ejemplo, un docente, un investigador o un analista), y en qué orden presentarlas.

En este ejercicio, te proponemos que, usando únicamente colecciones, diseñes una forma de:

1. Almacenar información básica de columnas.
2. Definir diferentes **roles** que indiquen qué columnas debe ver cada persona y cómo deben ordenarse.
3. Registrar en una [bitácora](#) todo aquello que te permita fundamentar las decisiones tomadas.

No se requiere leer archivos ni usar pandas. Todo el ejercicio se resuelve con Python básico y estructuras de datos. Consultar si se requiere el uso de otras librerías con el ayudante.

Columnas de trabajo

Trabajaremos con algunas columnas típicas de la **Encuesta Permanente de Hogares (Mercado Laboral)**. Estas son las columnas que usarás:

PONDERA, ESTADO, CAT_OCUP, EDAD, REGION, AGLOMERADO, AN04, TRIMESTRE, ITF, MAS_500, GDECCFR

Puedes asignarles los siguientes tipos (o inventar otros, pero mantené coherencia):

- PONDERA: int
- ESTADO: int (1=ocupado, 2=desocupado, 3=inactivo)
- CAT_OCUP: int (categoría ocupacional)
- EDAD: int
- REGION: int (región geográfica)
- AGLOMERADO: int
- MAS_500: string (Aglomerados según tamaño: N = Conjunto de aglomerados de menos de 500.000 habitantes, S = Conjunto de aglomerados de 500.000 y más habitantes)
- AN04: int (año)
- TRIMESTRE: int (1-4)

- `ITF`: int (ingreso total familiar)
- `GDECCFR`: int (grupo decil de ingreso)

De cada columna también se tiene el **porcentaje de completitud**, se debe completar con valores de 0 a 100%.

Consignas

1. Generará una estructura que almacene los datos de cada **columna**: nombre, el **tipo de dato** y el **porcentaje** de completitud de cada una.
2. Generará una estructura para almacenar al menos **3 roles diferentes** ('docente', 'investigador', 'analista'). Para cada rol debe definirse:
 - a. las columnas de interés.
 - b. el criterio por el que quiere que se ordene la información (**alfabéticamente** por nombre de columnas o por **porcentaje** de completitud).
 - c. la **forma** de ordenarlo (A: **ascendente**, B: **descendente**).
 - d. opcionalmente, un **porcentaje** mínimo de completitud, en caso que el rol esté configurado con esta opción. Si está presente, se deben mostrar las columnas de interés cuyo porcentaje de completitud sea **mayor o igual a ese valor**.
3. Creá un programa en Python que **informe** según el **rol** solicitado. Si no se especifica el rol debe informar **todas las columnas** ordenadas por **completitud** de forma **descendente**.
4. Completá con docstrings las funciones realizadas.
5. **Bitácora**: respondé las preguntas orientadoras (abajo) y registrá además **las decisiones de diseño** que tomaste, los errores que encontraste y cómo los resolviste.

Preguntas para registrar en la [Bitácora](#) y analizar al momento de escribir el código:

- ¿Qué ventajas tienen las estructuras elegidas para almacenar los datos de las columnas y roles con respecto a otras de las vistas en la teoría?
- ¿Qué valores elegiste para los roles y los porcentajes de completitud y por qué? ¿Cómo garantizaste que el programa pueda ser validado con diferentes roles, criterios de ordenamiento y umbrales?
- ¿Por qué conviene separar la configuración de los roles (`ROLES`) de la lógica que genera el informe?
- ¿Qué parámetros se pueden definir con valores por defecto?
- Si agregás una nueva columna al dataset, ¿en qué partes del código impacta? ¿Y si solo se quiere que un rol existente incluya esa nueva columna?
- ¿Qué pasaría si un rol tuviera un criterio de orden distinto a los especificados `"nombre"` o `"completitud"` (por ejemplo, `"promedio"`)? ¿Cómo lo detectarías y qué harías para que el programa no falle?

- ¿Qué cambiarías si por defecto se pide el informe debiera salir según uno de los roles?

Especificaciones Técnicas

- Se debe crear un **repositorio individual** en github.
- Se debe implementar **al menos una función**.
- Se valorará el uso de funciones como `map()`, `filter()`.
- El ejercicio deberá poder ejecutarse desde un **Jupyter notebook** dentro del proyecto.
- El código que tenga las funciones necesarias para la resolución deberán estar dentro del directorio `src`.
- Se debe también crear un `README.md` con tu nombre y legajo.
- Sube al repositorio la bitácora creada individualmente.

Video de defensa

Al iniciar la [actividad](#) vas a recibir **modificaciones** (tres preguntas) asignadas a vos y tendrás **una hora** para completar la tarea.

Plazo: La tarea estará disponible desde el 24/9 8:00h hasta el 28/9 23:59h inclusive.

Nota: tené en cuenta que la tarea se cierra automáticamente a las 23.59 del 28. Si la comenzás a las 23:30, solo tendrás hasta las 23:59; no se otorga tiempo extra.

1. Modificar el ejercicio según lo que te toque.
2. Grabar un **video breve (3–5 min)** donde muestres:
 - qué cambiaste y en qué parte del código;
 - por qué lo resolviste así;
 - cómo verificaste que funciona (caso de prueba concreto);
 - qué anotaste en la bitácora sobre ese cambio.
3. Subir el **código** y la bitácora actualizada al repositorio.
4. Al finalizar **subir el video al repositorio que prefieran** (YouTube, Drive, etc.).
5. **Poner** los links en la pregunta Refactor y buenas prácticas: del repositorio y del video en la tarea [“Actividad 1 práctica - modificaciones”](#) (**no olvidar dejar accesible el archivo para la corrección**).

Posibles herramientas para grabar:

- Pueden utilizar la herramienta que prefieran para su grabación: puede ser tomado desde el celular o algún capturador de pantalla como lo es [OBS](#), extensión de navegadores como Screen Recorder de Google Chrome, sitios como [Loom](#), por ejemplo.
-